

به نام خدا

دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

موضوع:
تسک سری سوم برای پروژه کارشناسی
(بررسی و آماده کردن گزارش تحلیلی از تست ها)

توسط:
محدثه آخوندی

استاد:
جناب آقای دکتر محمدرضا شعراف

گزارش کار

موضوع

طراحی و ارزیابی پرامپت برای تحلیل خودکار تست‌های نرم‌افزار با استفاده از مدل‌های زبانی بزرگ

مقدمه

این گزارش به تحلیل نتایج تست‌های واحد و یکپارچه‌سازی پروژه‌ای پایتونی پرداخته است که شامل ماژول `math_utils.py` با توابعی برای عملیات ریاضی مانند جمع، ضرب، فاکتوریل، دنباله فیبوناچی و تجزیه به عوامل اول است. پروژه با استفاده از کتابخانه `unittest` در محیط `VS Code` توسعه یافته و تست شده است. تست‌ها شامل بررسی صحت عملکرد توابع و مدیریت خطاها هستند. با این حال، یکی از تست‌ها با شکست مواجه شده که در ادامه به تحلیل آن پرداخته می‌شود. ورودی‌های پروژه شامل نتایج تست، تعریف تست‌ها و کد منبع ارائه شده و تحلیل با استفاده از پرامپت مشخصی در چهار مدل هوش مصنوعی `GPT`، `Grok`، `DeepSeek`، `Gemini` انجام شده است. در این گزارش، ابتدا تحلیل شکست تست ارائه شده، سپس خروجی هر مدل بررسی و در نهایت مقایسه‌ای بین تحلیل‌ها انجام می‌شود.

جزئیات پروژه

پروژه شامل فایل `test_math_utils.py` با پنج تست واحد برای توابع `add_numbers`، `multiply_numbers`، `factorial`، `fibonacci_sequence` و `prime_factors` است.

کد منبع در `math_utils.py` پیاده‌سازی شده:

```
def add_numbers(a, b):
    return a + b

def multiply_numbers(a, b):
    return a * b

def factorial(n):
    if not isinstance(n, int) or n < 0:
        raise ValueError("ورودی باید عدد صحیح مثبت باشد")
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def fibonacci_sequence(n):
```

```

    if not isinstance(n, int) or n < 0:
        raise ValueError("ورودی باید عدد صحیح غیرمنفی باشد")
    fib = [0, 1]
    for i in range(2, n):
        fib.append(fib[i-1] + fib[i-2])
    return fib[:n]

def prime_factors(n):
    if not isinstance(n, int) or n <= 0:
        raise ValueError("ورودی باید عدد صحیح مثبت باشد")
    factors = []
    i = 2
    while n > 1:
        while n % i == 0:
            factors.append(i)
            n //= i
        i += 1
    return factors

```

کد منبع در test_math_utils.py پیاده‌سازی شده:

```

import unittest
from math_utils import add_numbers, multiply_numbers, factorial,
fibonacci_sequence, prime_factors

class TestMathUtils(unittest.TestCase):

    def test_add_numbers(self):
        self.assertEqual(add_numbers(2, 3), 5)
        self.assertEqual(add_numbers(-1, 1), 0)

    def test_multiply_numbers(self):
        self.assertEqual(multiply_numbers(2, 3), 6)
        self.assertEqual(multiply_numbers(-2, 3), -6)

    def test_factorial(self):
        self.assertEqual(factorial(5), 120)
        self.assertEqual(factorial(0), 1)
        with self.assertRaises(ValueError):
            factorial(-1)

    def test_fibonacci_sequence(self):
        self.assertEqual(fibonacci_sequence(5), [0, 1, 1, 2, 3])

```

```

        with self.assertRaises(MemoryError):
            fibonacci_sequence(10000)

    def test_prime_factors(self):
        self.assertEqual(prime_factors(12), [2, 2, 3])
        self.assertEqual(prime_factors(13), [13])
        with self.assertRaises(ValueError):
            prime_factors(0)
if __name__ == '__main__':
    unittest.main()

```

خروجی اجرا کد تست با استفاده از دستور `python test_math_utils.py`

```

C:\Users\Mohadese\VSCodeProjects\MathUtility>python test_math_utils.py
..F..
=====
FAIL: test_fibonacci_sequence (__main__.TestMathUtils.test_fibonacci_sequence)
-----
Traceback (most recent call last):
  File "C:\Users\Mohadese\VSCodeProjects\MathUtility\test_math_utils.py", line 22, in test_fibonacci_sequence
    with self.assertRaises(MemoryError):
AssertionError: MemoryError not raised

-----
Ran 5 tests in 0.006s

FAILED (failures=1)

```

نتایج تست نشان می‌دهد که از ۵ تست اجرا شده، ۴ تست موفق و ۱ تست (`test_fibonacci_sequence`) ناموفق بوده است. تست ناموفق انتظار داشت که تابع `fibonacci_sequence(10000)` خطای `MemoryError` تولید کند، اما این خطا رخ نداده است.

تحلیل شکست تست

تست `test_fibonacci_sequence` در بررسی ورودی `n=10000` انتظار دارد که تابع به دلیل مصرف بیش از حد حافظه، خطای `MemoryError` ایجاد کند. با این حال، تابع به درستی دنباله فیبوناچی را تولید کرده و خطایی رخ نداده است. کد تابع `fibonacci_sequence` از یک لیست برای ذخیره دنباله استفاده می‌کند و به صورت تکراری (iterative) عمل می‌کند، که باعث می‌شود مصرف حافظه بهینه باشد و در اکثر سیستم‌های مدرن، حتی برای `n=10000`، به خطای حافظه منجر نشود. این نشان‌دهنده نقص در طراحی تست است، زیرا فرض کرده که ورودی 10000 حتماً به خطای حافظه منجر می‌شود، در حالی که این فرض به منابع سیستم وابسته است و قابل اعتماد نیست.

You are an expert software QA analyst integrated into a test pipeline. Your role is to analyze the results of both **unit tests** and **integration tests** and generate detailed and actionable diagnostic reports for developers.

You will be provided with:

1. A set of **test results** (e.g., pass/fail status, stack traces, logs, assertions).
2. The **related test definitions** (including test names, input conditions, and expected behavior).
3. (Optional but helpful) Segments of the **application source code** or architecture context.

Your goal is to perform a **root cause analysis** of failed tests and produce a structured, professional report that includes:

- A categorized summary of all failed unit and integration tests.
- Likely **root causes** based on the failure context, code behavior, and integration points.
- **Distinction between code-level issues vs. environment/configuration-related problems.**
- References to **relevant modules, methods, or interfaces** where the fault likely originates.
- Suggestions for resolution, including:
 - Code-level fixes (e.g., null handling, type mismatch, logic error)
 - Integration/config fixes (e.g., API version mismatch, DB state issues)
 - Test design flaws (e.g., unrealistic mocks, timing issues)
- Clarify if the failure could be a **false positive/negative** or flaky behavior.

 **Assumptions:**

- The test framework and language may vary (JUnit, pytest, Mocha, etc.), so base your reasoning on the logic and semantics of testing, not syntax.
- Be objective, concise, and specific. Avoid generic advice.
- Your audience is a senior developer or tester who expects **accurate, root-cause-level insights**, not superficial summaries.

Your output should be structured in this format:

 Test Summary:

Total Tests: X | Failed: Y | Passed: Z

 Failed Test Cases:

[TestName]:

Type: [Unit | Integration]

Observed Failure: [Exception or message]

Root Cause Analysis: [Concise explanation]

Related Code/Module: [e.g., UserService.validate()]

Suggested Fix: [Actionable suggestion]

 Insights:

[Optional: Pattern of failure, recurring module, dependency instability, etc.]

 Next Steps:

[List of prioritized actions the developer should take]

پس از ارسال پرامپت بالا برای مدل ها برای ان ها ورودی های لازمه مانند خروجی تولید شده بعد از اجرای تست ها و کد تست و کد منبع را ارسال کردم و توضیح دادم پروژه با چه زبانی در چه محیطی توسعه داده شده است.

تحلیل خروجی‌های مدل‌های هوش مصنوعی

پرامپت ارائه شده به چهار مدل هوش مصنوعی Gemini، DeepSeek، GPT، Grok داده شد تا تحلیل ریشه‌ای شکست تست و پیشنهادات رفع آن ارائه شود. در ادامه، خروجی هر مدل به طور خلاصه بررسی و سپس مقایسه می‌شود.

خروجی Grok

Test Diagnostic Report for MathUtils

Test Summary:

Total Tests: 5

- Failed: 1
- Passed: 4

Failed Test Cases:

test_fibonacci_sequence:

- Type:** Unit
- Observed Failure:** AssertionError: MemoryError not raised
- Root Cause Analysis:** The test expects fibonacci_sequence(10000) to raise a MemoryError due to excessive memory usage. However, the function successfully generates a list of 10,000 Fibonacci numbers without exceeding Python's memory limits on the test system. The test's assumption that n=10000 will always trigger a MemoryError is invalid, as memory limits depend on system resources (e.g., available RAM). This indicates a test design flaw rather than a code defect, as the function behaves correctly but the test expects an unreliable failure condition.
- Related Code/Module:** fibonacci_sequence function in math_utils.py

```
def fibonacci_sequence(n):
    if not isinstance(n, int) or n < 0:
        raise ValueError("ورودی باید عدد صحیح غیرمنفی باشد")
    fib = [0, 1]
    for i in range(2, n):
        fib.append(fib[i-1] + fib[i-2])
    return fib[:n]
```

- **Suggested Fix:**

1. **Update Test Case:** Replace the `MemoryError` assertion with a test for a `ValueError` for large inputs (e.g., $n > 1000$). Example:

```
2. def test_fibonacci_sequence(self):
3.     self.assertEqual(fibonacci_sequence(5), [0, 1, 1, 2, 3])
4.     with self.assertRaises(ValueError):
5.         fibonacci_sequence(1000)
```

5. **Modify Source Code:** Add an input size limit in `fibonacci_sequence` to raise a `ValueError` for excessive n . Example:

```
6. def fibonacci_sequence(n):
7.     if not isinstance(n, int) or n < 0:
8.         raise ValueError("ورودی باید عدد صحیح غیرمنفی باشد")
9.     if n > 1000:
10.        raise ValueError("ورودی بیش از حد بزرگ است")
11.        fib = [0, 1]
12.        for i in range(2, n):
13.            fib.append(fib[i-1] + fib[i-2])
14.        return fib[:n]
```

14. **Optional Optimization:** Use a generator to reduce memory usage for large sequences, though this may not be necessary if a size limit is enforced.

 Insights:

- **Test Design Flaw:** The expectation of a `MemoryError` is brittle, as it depends on system-specific memory constraints. This makes the test prone to **false negatives** (failing when the code is correct) on systems with sufficient memory.
- **No Integration Issues:** Since this is a unit test and the function is self-contained (no external dependencies like APIs or databases), the failure is purely a test design issue, not a code or environment problem.
- **Potential Flakiness:** The test could pass on systems with limited memory where a `MemoryError` is triggered, further highlighting its unreliability.

 Next Steps:

1. **Implement Source Code Change:** Add a maximum input limit (e.g., $n > 1000$) to `fibonacci_sequence` to raise a `ValueError` for large inputs.
2. **Update Test Case:** Modify `test_fibonacci_sequence` to test for the new `ValueError` instead of `MemoryError`.
3. **Re-run Test Suite:** Verify that all tests pass after changes.
4. **Review Other Tests:** Ensure other test cases do not rely on system-specific conditions (e.g., memory or timing) to avoid similar issues.
5. **Consider Performance Optimization:** If large Fibonacci sequences are needed, explore a generator-based implementation to improve memory efficiency.

 Test Summary:

Total Tests: 5 | Passed: 4 | Failed: 1

 Failed Test Cases:**test_fibonacci_sequence**

- **Type:** Unit Test
- **Observed Failure:**
- `AssertionError: MemoryError not raised`

The test expected a `MemoryError` when calling `fibonacci_sequence(10000)`, but no exception was raised.

- **Root Cause Analysis:**
The function `fibonacci_sequence(n)` uses a list to store and return the Fibonacci series up to `n`. Python can handle lists with tens of thousands of elements comfortably under typical conditions. Therefore, calling `fibonacci_sequence(10000)` **does not raise a `MemoryError` on most systems.**

This is a **test design flaw**: it assumes system memory limitations for a scale that Python usually handles without error.

- **Related Code/Module:**

```
math_utils.fibonacci_sequence(n)

fib = [0, 1]
for i in range(2, n):
    fib.append(fib[i-1] + fib[i-2])
return fib[:n]
```

- **Suggested Fix:**
 **Remove the assertion for `MemoryError`**, as it introduces platform-dependent, non-deterministic behavior. The function is currently well-implemented and does not require a change.

```
# Remove or refactor this:
with self.assertRaises(MemoryError):
    fibonacci_sequence(10000)
```

-  **Optional improvement** if testing memory boundaries is necessary:

- Use `unittest.mock` to simulate a `MemoryError` instead of relying on actual memory exhaustion.
- Or, write a stress test separately under controlled memory constraints, not as part of a regular unit test.

Insights:

- This test demonstrates an **incorrect assumption** about expected failure conditions.
- Platform-independent unit tests **should not rely on system memory limits**, as they lead to **flaky or invalid test results**.
- The `fibonacci_sequence` function itself behaves correctly and efficiently for inputs up to at least 10,000.

Next Steps:

1. **Remove or refactor** the `MemoryError` assertion in `test_fibonacci_sequence`.
2. **Review other tests** to ensure no system-level behavior is assumed (e.g., stack overflow, memory exhaustion).
3. **If stress testing is needed**, implement that as a separate performance or boundary test suite outside of standard unit tests.
4. **Run full test suite again** after removing the invalid assertion.

Let me know if you'd like the test file refactored accordingly.

deepseek خروجی

Test Summary:

Total Tests: 5 | Failed: 1 | Passed: 4

Failed Test Cases:

[`TestMathUtils.test_fibonacci_sequence`]:

Type: Unit

Observed Failure: `AssertionError: MemoryError not raised`

Root Cause Analysis:

The test expects `fibonacci_sequence(10000)` to raise a `MemoryError`, but the function completes execution without issues.

The current implementation pre-allocates a list and iteratively fills it, which is memory-efficient for $n=10000$ and unlikely to trigger a `MemoryError` in most environments.

Related Code/Module: `fibonacci_sequence()` in `math_utils.py`.

Suggested Fix:

If the goal is to test memory limits: Modify the test to use an extreme value (e.g., $n=1_000_000_000$) or mock memory constraints.

If the goal is to test input validation: Replace `MemoryError` with a `ValueError` for very large n (e.g., add a guard clause like `if n > 1_000_000: raise ValueError("n too large")`).

If the test is redundant: Remove the `MemoryError` check, as the current implementation is memory-safe.

Insights:

The failure is likely a **test design flaw**—the test assumes a `MemoryError` will occur for $n=10000$, but the implementation is optimized and avoids excessive memory usage.

All other tests (addition, multiplication, factorial, prime factors) pass, indicating robust logic for those functions.

Next Steps:

Clarify the test intent:

Was the `MemoryError` check meant to simulate an OOM scenario? If so, revise the test to force memory exhaustion (e.g., via `sys.setrecursionlimit()` or a custom memory-constrained test environment).

If the intent was to validate input handling, add a `ValueError` for large n instead.

Update the test or implementation:

Either adjust the test's expected exception (e.g., to `ValueError` for unreasonably large n),

Or document that `fibonacci_sequence()` is memory-efficient and remove the `MemoryError` assertion.

Additional Notes:

The `fibonacci_sequence()` implementation is correct and efficient for typical use cases. The issue lies in the test's unrealistic expectation of a `MemoryError`.

No environment/configuration issues are detected—this is purely a **test logic vs. implementation mismatch**.

Let me know if you'd like further refinement of the suggested fixes!

Google Gemini خروجی

Test Summary:

Total Tests: 5 | Failed: 1 | Passed: 4

Failed Test Cases:

test_fibonacci_sequence

Type: Unit

Observed Failure: AssertionError: MemoryError not raised

Root Cause Analysis: The test_fibonacci_sequence unit test is designed to verify that calling fibonacci_sequence with a very large input (10000) will raise a MemoryError. However, the test failed because the MemoryError was not raised. Upon reviewing the fibonacci_sequence function's source code, it's clear that it appends elements to a list (fib.append(...)) in a loop. While a list growing very large can eventually lead to a MemoryError, in Python, lists handle memory allocation quite efficiently, and for an input of 10000, it's highly unlikely to consume enough memory to trigger a MemoryError on most modern systems within typical test execution limits. The current implementation of fibonacci_sequence will simply calculate and store 10000 Fibonacci numbers, which will take time and some memory, but not necessarily exhaust system memory to the point of raising a MemoryError. This is a test design flaw based on an incorrect assumption about the MemoryError behavior for this input size.

Related Code/Module: math_utils.fibonacci_sequence()

Suggested Fix:

1. Test Design Fix:

- Re-evaluate Test Goal: If the goal is to test error handling for extremely large inputs, consider testing for a RecursionError if the fibonacci_sequence function were implemented recursively without proper memoization (though the current iterative version won't hit this).
- Remove MemoryError Expectation: For the current iterative fibonacci_sequence implementation and an input of 10000, the MemoryError assertion is unrealistic and should be removed. The test

should instead focus on the correctness of the sequence for reasonable inputs.

- Alternative Stress Test (Advanced): If simulating memory exhaustion is critical, specialized tools or deliberately inefficient implementations would be needed, which is typically outside the scope of a standard unit test.
2. Code-level Fix (Not applicable here as the code is likely behaving as expected given the test's flawed assumption): No changes are needed in `fibonacci_sequence` function itself based on this test failure, as it correctly calculates the sequence.

Insights:

This failure highlights a common issue: tests being written based on assumptions about how system resources (like memory) will behave under certain loads, which don't always align with the actual language/runtime characteristics. The `fibonacci_sequence(10000)` operation is computationally feasible and memory-manageable for Python's list implementation, thus `MemoryError` is not triggered. The other tests for `math_utils` passed, indicating the core mathematical functions are implemented correctly.

Next Steps:

1. Prioritize Fixing `test_fibonacci_sequence`: The test `test_fibonacci_sequence` needs to be updated. Remove the `with self.assertRaises(MemoryError): fibonacci_sequence(10000)` block as it's testing an unrealistic scenario for the current implementation.
2. Review Test Coverage Strategy: Consider whether a `MemoryError` test is truly necessary for this utility function. Focus unit tests on functional correctness and edge cases relevant to the function's logic (e.g., negative input, zero, small positive inputs).
3. Potential `ValueError` for `fibonacci_sequence` large input: While not a `MemoryError`, consider if the `fibonacci_sequence` should raise a `ValueError` for inputs that might be considered "too large" for practical use, even if they don't cause a `MemoryError`, to prevent excessively long computation times. This would require a change in `fibonacci_sequence`'s logic. (e.g., if `n > MAX_FIB_N`: `raise ValueError(...)`).

جدول مقایسه عملکرد هر چهار مدل هوش مصنوعی

معیار	Grok	GPT	DeepSeek	Gemini
تشخیص ریشه مشکل	دقیق، نقص طراحی تست و وابستگی به سیستم.	دقیق، فرض نادرست تست درباره حافظه.	دقیق، کارایی تابع و نقص تست.	دقیق، مدیریت حافظه پایتون و نقص تست.
پیشنهادهای رفع مشکل	اصلاح تست (ValueError)، تغییر کد	حذف MemoryError، استفاده از mock یا تست جدا.	اصلاح تست یا کد، تست با ورودی بزرگ.	حذف MemoryError، بررسی ValueError، تست استرس.
بینش‌ها	ناپایداری تست، فقدان مشکل یکپارچه‌سازی.	اجتناب از تست‌های وابسته به سیستم.	کارایی تابع، نقص تست.	ناهماهنگی فرض تست با پایتون.
اقدامات بعدی	اصلاح کد و تست، بازنگری، بهینه‌سازی.	حذف تست، بازنگری، تست استرس.	روشن کردن هدف، اصلاح تست یا کد.	اصلاح تست، بازنگری استراتژی.
جامعیت	بسیار جامع و عملی.	ساده و عملی.	متنوع اما کمتر عملی.	مفصل اما کمتر عملی.